

Laboratorio 13 · Monitorización con Percona Monitoring and Management y alertas

Duración: 30 minutos · Carpeta de trabajo: `entorno/`

Objetivos

Vas a desplegar PMM 3, a registrar los tres nodos del replica set, a interpretar los paneles principales de MongoDB, a encontrar las consultas más costosas con Query Analytics y a crear una alerta que salte cuando un nodo cae.

Requisitos

PMM Server necesita unos 2-3 GB de RAM. Si tu equipo va justo, para PBM (`docker compose -f compose.yml -f compose.pbm.yml stop pbm1 pbm2 pbm3`) y cierra aplicaciones que no uses.

Paso 1 · Usuario de monitorización en MongoDB

```
cd entorno
docker compose up -d
docker compose exec toolbox bash
mongosh "$RS"
```

```
> use admin
> db.createRole({
  role: "pmmMonitor",
  privileges: [
    { resource: { db: "", collection: "" }, actions: [ "dbHash",
"find", "listIndexes", "listCollections", "collStats", "dbStats",
"indexStats" ] },
    { resource: { db: "", collection: "system.version" }, actions: [
"find" ] },
    { resource: { db: "", collection: "system.profile" }, actions: [
"find", "dbStats", "collStats", "indexStats" ] }
  ],
  roles: []
})
```

```

    })
    > db.createUser({
      user: "pmm", pwd: "PmmCurso2026",
      roles: [
        { db: "admin", role: "pmmMonitor" },
        { db: "local", role: "read" },
        { db: "admin", role: "clusterMonitor" },
        { db: "admin", role: "directShardOperations" }
      ]
    })
    > exit

```

Paso 2 · Activar el profiler en los tres nodos

Query Analytics necesita una fuente de consultas. Usaremos el **profiler** de MongoDB, que guarda en `system.profile` las operaciones más lentas que un umbral. El profiler es una configuración **por nodo** que no se replica, así que hay que activarlo en cada uno:

```

for h in mongo1 mongo2 mongo3; do
  mongosh "mongodb://admin:CursoMongo2026@$h:27017/tienda?
authSource=admin&directConnection=true" --quiet \
    --eval 'printjson(db.setProfilingLevel(1, { slowms: 50 })))'
done
exit

```

Paso 3 · Arrancar PMM y registrar los nodos

```

docker compose -f compose.yml -f compose.pmm.yml up -d
docker logs -f pmm-client

```

Espera a ver en el log del cliente que se ha registrado correctamente en el servidor (puede tardar uno o dos minutos mientras PMM Server arranca; el cliente se reinicia solo hasta conseguirlo). Pulsa Ctrl+C y registra los tres nodos:

```

docker exec pmm-client pmm-admin add mongodb --username=pmm --
password=PmmCurso2026 --host=mongo1 --port=27017 --service-name=mongo1 -
-cluster=rs0 --replication-set=rs0 --query-source=profiler --enable-all-
collectors
docker exec pmm-client pmm-admin add mongodb --username=pmm --
password=PmmCurso2026 --host=mongo2 --port=27017 --service-name=mongo2 -
-cluster=rs0 --replication-set=rs0 --query-source=profiler --enable-all-
collectors

```

```
docker exec pmm-client pmm-admin add mongodb --username=pmm --  
password=PmmCurso2026 --host=mongo3 --port=27017 --service-name=mongo3 -  
-cluster=rs0 --replication-set=rs0 --query-source=profiler --enable-all-  
collectors  
docker exec pmm-client pmm-admin list
```

Paso 4 · Generar carga

En otra terminal, deja corriendo la carga de trabajo durante el resto del laboratorio:

```
docker compose exec toolbox bash  
mongosh "$RS" --quiet --eval "var MINUTOS=20" /scripts/carga.js
```

Paso 5 · Explorar los paneles

Abre <https://localhost:8443> en el navegador, acepta el aviso del certificado autofirmado y entra con `admin` / `CursoMongo2026`. Busca en el menú de MongoDB estos paneles (los nombres exactos pueden variar ligeramente entre versiones de PMM 3):

1. **MongoDB Instances Overview / Summary:** operaciones por segundo, conexiones, uso de la caché de WiredTiger.
2. **MongoDB ReplSet Summary:** estado de cada miembro, lag de replicación, ventana del oplog, elecciones.
3. **MongoDB WiredTiger Details:** caché, *tickets* de lectura y escritura, checkpoints.

Responde con lo que veas:

1. ¿Cuántas operaciones por segundo de cada tipo (query, insert, update) está recibiendo el primario?
2. ¿Cuál es el lag de replicación de cada secundario? ¿Cuánto tiempo cubre el oplog?
3. ¿Qué porcentaje de la caché de WiredTiger está en uso?

Paso 6 · Query Analytics

Entra en **Query Analytics (QAN)**, filtra por el servicio del primario y ordena por *Load* o *Query Time*.

1. ¿Cuál es la consulta que más carga genera?
2. Abre su detalle: ¿cuántos documentos examina frente a cuántos devuelve?
3. Mira la pestaña **Explain**. ¿Qué plan usa?

▼ Ver solución

Las consultas de `carga.js` sobre pedidos filtrando por `cliente_id` o por `estado + ciudad` no tienen índice: examinan los 200.000 documentos (`COLLSCAN`) para devolver unos pocos. En QAN aparecen arriba del todo con una relación *docs examined / docs returned* altísima. Es exactamente el tipo de problema que resolveremos en el laboratorio 14 creando los índices adecuados.

Paso 7 · Una alerta que salte cuando cae un nodo

PMM 3 usa el sistema de alertas de Grafana y trae plantillas predefinidas. Ve a **Alerting** → **Alert rule templates**, busca la plantilla para "MongoDB down" (o similar) y crea una regla a partir de ella con estos valores:

- Duración (*for*): 1 minuto.
- Severidad: crítica.
- Filtro: servicio del replica set `rs0` (o sin filtro para vigilar todos).

En un entorno real configurarías también un **punto de contacto** (*contact point*: correo, Slack, Teams, PagerDuty...) y una política de notificación. En el laboratorio basta con ver la alerta en la interfaz.

Ahora provoca la caída de un secundario:

```
docker compose stop mongo3
```

Vigila **Alerting** → **Alert rules**: la regla pasará de *Normal* a *Pending* y, pasado el minuto, a *Firing*. Mira también el panel **ReplSet Summary**. Después recupera el nodo y observa cómo la alerta vuelve a *Normal*:

```
docker compose start mongo3
```

Para pensar: ¿qué alertas pondrías en producción?

▼ Una propuesta de mínimos

Alerta	Condición orientativa	Por qué
Nodo caído	Instancia no responde 1 min	Pérdida de redundancia
Sin primario	Ningún miembro PRIMARY 30 s	La aplicación no puede escribir
Lag de replicación	> 30-60 s durante 5 min	Riesgo de pérdida de datos en failover y lecturas obsoletas
Ventana del oplog	< 24 h (o menos de tu ventana de mantenimiento)	Un secundario caído necesitaría resincronización completa

Alerta	Condición orientativa	Por qué
Conexiones	> 80 % del máximo	Pool de conexiones mal dimensionado o fuga
Caché WiredTiger	Páginas sucias > 20 % o desalojo por hilos de aplicación	El disco no da abasto; latencias altas
Disco	> 80 % ocupado	MongoDB no libera espacio al borrar; el crecimiento es continuo
Certificado TLS	Caduca en < 30 días	Evitar una parada completa por certificado caducado
Backup	Último backup correcto hace > 26 h	Detectar backups que fallan en silencio

Para terminar

PMM consume bastante memoria. Páralo cuando termines:

```
docker compose -f compose.yml -f compose.pmm.yml stop pmm-server pmm-client
```

Para la carga de la otra terminal con Ctrl+C si sigue en marcha.